Trabalho Prático: Tratamento de Exceções, Pilha de Execução e Regras de Negócio em Java

1. Contextualização

Dando continuidade ao sistema de folha de pagamento e processamento de contas (Payable), a empresa identificou a necessidade de fortalecer os mecanismos de auditoria, conformidade trabalhista e integridade transacional. Operações financeiras e registros de jornada de trabalho não podem falhar silenciosamente nem gerar inconsistências de estado no sistema.

Nesta atividade, você irá refatorar e expandir o modelo para implementar controle rigoroso de fluxo com **exceções customizadas (*checked* e *unchecked*)**, propagação pela **pilha de chamadas (*call stack*)**, manipulação com blocos try-catch-finally e estrutura de **pilha de auditoria/rollback** para estorno de transações com falha.

2. Objetivos de Aprendizagem

- Projetar hierarquias de exceções personalizadas estendendo Exception (*checked*) e RuntimeException (*unchecked*).
- Dominar as cláusulas throws, throw, blocos try-catch-finally e encadeamento de exceções (*exception chaining / cause*).
- Analisar a pilha de execução (*stack trace*) para depuração e rastreabilidade de erros em múltiplos níveis de chamadas.
- Utilizar uma estrutura de dados de Pilha (java.util.ArrayDeque ou java.util.Stack) para rastrear execuções e possibilitar a reversão (*rollback*) de transações financeiras em lote caso ocorra um erro de processamento.

3. Especificação dos Requisitos

Parte 1: Hierarquia de Exceções Customizadas

1. **ExcessoJornadaException (Checked Exception - estende Exception):**
   - Deve ser disparada sempre que o registro diário de horas de um funcionário horista (HourlyEmployee) ultrapassar o limite legal de 2 horas extras (ou seja, mais de 10 horas totais em um único dia).
   - O construtor deve armazenar e fornecer métodos de acesso para: nome do colaborador, data/dia do registro, quantidade de horas lançadas e limite legal permitido.
2. **DadosInvalidosException (Unchecked Exception - estende IllegalArgumentException):**
   - Disparada em validações básicas de domínio (ex.: valor de hora negativo,

porcentagem de comissão fora da faixa de 0 a 100%, salário base negativo, fatura com valor negativo ou quantidade zero/negativa).

3. **FalhaProcessamentoFolhaException (Checked Exception - estende Exception):**
   - Exceção de alto nível lançada pela controladora da folha (PayrollService). Deve encapsular a exceção original (usando o conceito de causa raiz: super(mensagem, causa)).

Parte 2: Lançamento e Propagação na Pilha de Chamadas

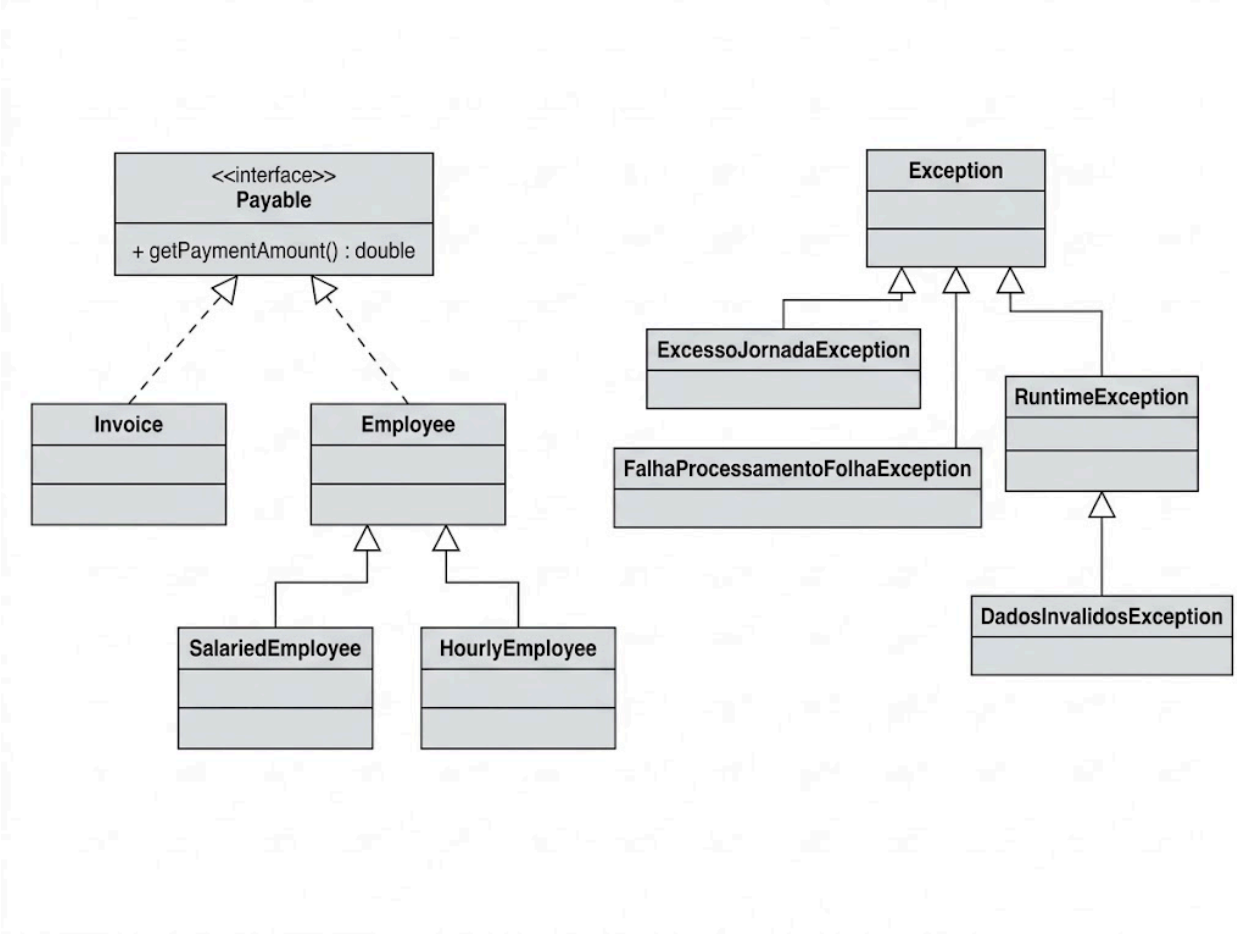
1. **Modelagem das Classes de Domínio:**
   - **HourlyEmployee:** Ao registrar a folha ou lançar as horas trabalhadas da semana/dia, o método deve validar a carga diária. Caso a jornada exceda 10 horas/dia (8 normais + 2 extras), o método **não deve engolir o erro com catch**, e sim declará-lo com throws ExcessoJornadaException e lançá-lo via throw.
   - **Invoice:** Ao instanciar ou alterar itens, quantidades menores ou iguais a zero devem disparar DadosInvalidosException.
2. **Propagação em Níveis de Pilha:**
   - Construa uma arquitetura em 3 camadas:
     - **Nível 1 (Entidade/Domínio):** Valida e dispara as exceções (throw).
     - **Nível 2 (Serviço/Regra de Negócio - PayrollService):** Processa lotes de pagamentos. Ao capturar uma ExcessoJornadaException, registra o apontamento para auditoria trabalhista e relança como FalhaProcessamentoFolhaException encapsulando a causa.
     - **Nível 3 (Visão/Main):** Ponto de entrada que captura as falhas finais, imprime o stackTrace amigável no console e exibe o estado final do sistema.

Parte 3: Gerenciamento com Estrutura de Pilha (Pilha de Transações / Rollback)

1. Crie uma classe utilitária de controle transacional: HistoricoTransacoes.
2. Essa classe deve conter internamente uma estrutura de **Pilha** (Deque<Payable> ou Stack<Payable>).
3. Ao processar um lote de pagamentos (Payable[]):
   - Cada item pago com sucesso deve ser empilhado (push) no histórico.
   - Se o processamento de algum item da lista lançar uma exceção no meio do lote, o bloco catch deve acionar uma rotina de reversão (*rollback*).
   - A rotina de reversão deve desempilhar (pop) todos os pagamentos realizados até o momento da falha, executando o estorno de cada um e exibindo a ordem LIFO (*Last-In, First-Out*) em que foram cancelados.
4. Utilize obrigatoriamente um bloco finally para fechar o ciclo de auditoria (ex.:

emitir o resumo final informando quantas operações foram concluídas, quantas foram estornadas e garantir que os recursos de log sejam descarregados).

4. Diagrama de Classes Sugerido



5. Cenário de Teste Obrigatório no Main

O método principal (public static void main(String[] args)) deve demonstrar os três cenários:

1. **Caminho Feliz:** Processamento de pagamentos válidos contendo Invoice, SalariedEmployee e HourlyEmployee dentro do limite legal[cite: 1].
2. **Cenário de Violação Trabalhista (Checked):** Submissão de um HourlyEmployee com registro diário de 12 horas[cite: 1]. O sistema deve disparar ExcessoJornadaException, capturá-la no serviço, registrar a pendência no relatório de auditoria e exibir a pilha de chamadas (*stack trace*)[cite: 1].

3. **Cenário de Falha com Rollback (Pilha):** Processamento em lote de 5 pagamentos onde o 4º item gera uma falha. O sistema deve demonstrar os 3 primeiros pagamentos sendo empilhados e, em seguida, desempilhados um a um para estorno (*rollback*), seguido da mensagem do bloco finally.